# Sri Lanka Institute of Information Technology


## Final Project Report: Sakura Saki Beauty Salon Appointment System

**Course Code:** SE1020

**Date:** May 28, 2026

Batch: Y1S2


| Name | Student ID |
|---|---|
| Perera W. P. U. M | IT25103877 |
| Athurugiriya A. A. A. A. Y. | IT25103878 |
| Vimansa K. B. A. O. | IT25103879 |
| Fernando S. J. S. | IT25103880 |
| Janandhana M. H. C | IT25103881 |
| Malinga P. D. | IT25103882 |

# Table of Contents

## 1. Executive Summary

**Sakura Saki** (桜咲き, which translates to "Sakura bloom" in English) is an integrated, web based beauty salon management system engineered to digitalize, centralize, and significantly streamline core salon operations. Historically, salon management has relied on disparate tools or manual ledgers for appointments, staff scheduling, and customer records, often leading to inefficiencies and scheduling conflicts. Sakura Saki directly addresses these industry pain points by consolidating customer account management, dynamic service catalogs, conflict free appointment scheduling, staff tracking, and post-visit review moderation into a single, cohesive platform.

The project's visual identity is deeply tied to its name, featuring an immersive 3D and animated Sakura theme that provides a calming, premium user experience. This aesthetic focus ensures that the software is not only functionally robust but also highly engaging for end-users. Under the hood, the system is strictly built upon robust Object-Oriented Programming (OOP) principles and leverages modern Java web technologies alongside a comprehensive CI/CD pipeline and cloud infrastructure. By bridging intuitive frontend design with a highly scalable, secure, and maintainable backend enterprise architecture, Sakura Saki delivers a complete end-to-end digital transformation for modern beauty salons.

## 2. Project Overview & Objectives

The primary objective of the Sakura Saki project is to transition traditional, manual salon management into a fully automated digital ecosystem.
Key objectives include:

- Establishing a seamless appointment booking and scheduling workflow.
- Centralizing user management for customers, staff, and administrators.
- Implementing a dynamic catalog for individual salon services and bundled packages.
- Creating a feedback loop through an integrated, moderated review system.
- Designing a scalable backend architecture utilizing modular development and extensive CRUD functionalities.

## 3. System Architecture

The application employs an N-tier, layered architectural pattern utilizing the Spring Boot framework to ensure separation of concerns and maintainability:

- **Entity/Model Layer**: Domain-driven Java classes mapped directly to database tables via JPA/Hibernate.
- **Repository Layer**: Data Access Object (DAO) interfaces utilizing Spring Data JPA for optimized database interactions.
- **Service Layer**: Encapsulates core business logic, transaction management, and validation rules.
- **Controller Layer**: RESTful and MVC controllers handling HTTP requests, routing, and connecting the backend to the presentation layer.
- **View Layer**: Thymeleaf templates responsible for server-side rendering of dynamic HTML forms, dashboards, and reports.

## 4. Technology Stack & Key Characteristics

The system leverages a modern, industry-standard technology stack:

- **Backend Framework (Java & Spring Boot)**: Delivers an embedded server, rapid MVC development, Dependency Injection (IoC), and highly scalable REST capabilities.
- **Security (Spring Security)**: Enforces robust, session-based authentication and Role-Based Access Control (RBAC) to secure sensitive administrative endpoints.
- **Database & ORM (MySQL & Hibernate)**: MySQL provides reliable relational data storage, while Spring Data JPA/Hibernate handles Object-Relational Mapping, abstracting complex SQL queries.
- **Frontend Presentation (Thymeleaf, HTML, CSS, Bootstrap, JS)**: Thymeleaf seamlessly binds backend data to the UI. Bootstrap ensures a responsive, mobile-first design, and JavaScript handles DOM interactivity.
- **Interactive 3D UI Design (Three.js & CSS Animations)**: The frontend embraces the "Sakura Saki" theme through dynamic 3D graphical effects (such as animated, falling cherry blossom petals) rendered via Three.js. This immersive and animated UI design not only aligns with the brand identity but also creates a premium, engaging aesthetic for salon customers.
- **Build & Dependency Management (Maven)**: Automates the build lifecycle and manages library dependencies consistently.
- **DevOps & Cloud Infrastructure**:
    - **Docker**: Containerization using multi-stage builds.
    - **Render**: Infrastructure as Code (`render.yaml`) for cloud application hosting.
    - **Aiven Cloud**: Fully managed, cloud-hosted MySQL database cluster.
    - **GitHub Actions**: Automated CI/CD pipelines for deployment and testing.
    - **Postman**: Comprehensive API testing and collection management.

## 5. Stakeholder Analysis & Functional Requirements

### 5.1 Customers

Clients utilizing the platform to interact with salon services.

- **Authentication**: Secure registration, login, and session management.
- **Profile Management**: Maintain and update personal contact information and credentials.
- **Service Browsing**: Search, filter, and view detailed information on available services and bundled packages.
- **Appointment Booking**: Request new bookings, view upcoming itineraries, and seamlessly reschedule or cancel appointments.
- **Reviews**: Provide post-visit ratings and feedback; manage personal review history.

### 5.2 Administrators

System operators responsible for business configuration and oversight.

- **Access Control**: Secure login into protected administrative routing.
- **User Management**: Search, filter, deactivate, or delete customer accounts; provision new admin roles.
- **Catalog Management**: Execute full CRUD operations on individual salon services and service packages, including dynamic pricing and discount configurations.
- **Staff Management**: Register beauticians, assign specialized roles (e.g., Stylist, Therapist), and manage working schedules.
- **Reporting & Dashboards**: Monitor key performance indicators (KPIs) such as total registered users, active services, staff count, and daily appointment volumes.
- **Content Moderation**: Review, hide, or permanently remove inappropriate customer feedback.

### 5.3 Staff / Beauticians

Salon employees providing specialized beauty services.

- **Profile & Specialty Tracking**: Represented in the system with specific roles and skill sets.
- **Schedule Integrity**: Systemic tracking of available time slots and working days to prevent scheduling conflicts and double-booking.

## 6. Database Design & Implemented Entities

The relational database schema is strictly mapped to the following Java domain models using JPA annotations:

- **User**: The foundational base entity for authentication, storing shared credentials and role definitions.
- **Admin**: Inherits from User; represents users with elevated system privileges.
- **Customer**: Inherits from User; represents the salon's clients.
- **Staff**: An abstract base entity representing salon employees and their core attributes.
- **Stylist & Therapist**: Concrete, specialized entities inheriting from Staff, differentiating employee capabilities.
- **SalonService**: Represents distinct, individual beauty services offered by the salon.
- **ServicePackage**: Represents bundled collections of services, typically offered at a discounted rate.
- **Appointment**: The transactional entity linking a Customer, a Staff member, and a SalonService or ServicePackage for a specific timestamp.
- **Review**: Stores customer feedback, ratings, and moderation statuses linked to specific services or appointments.

## 7. Object-Oriented Principles Applied

The backend architecture rigorously adheres to fundamental OOP principles:

- **Encapsulation**: State is protected within entity classes. Fields are explicitly marked as `private` and mutated exclusively via controlled getters, setters, and dedicated service-layer methods, preventing unauthorized data modification.
- **Inheritance**: Extensively utilized to enforce DRY (Don't Repeat Yourself) principles. The `User` base class is extended by `Admin` and `Customer`. Similarly, the `Staff` class acts as a superclass for `Stylist` and `Therapist`.
- **Abstraction**: Business logic is abstracted behind interfaces (e.g., `AppointmentService`), decoupling the controller layer from the underlying data access and query logic. Abstract classes (like `Staff`) define contractual behaviors without exposing implementation details.
- **Polymorphism**: Implemented across service layers and UI rendering. For instance, authentication mechanisms dynamically handle `User` objects, routing them to different dashboards based on their runtime polymorphic type (Admin vs. Customer).

# 8. Workload Distribution & Module Ownership

### 8.1 Module 1: Customer & Authentication

- **Responsibility**: End-to-end customer identity management and access control.
- **CRUD Operations**:

  Customer registration (Create)

  Profile viewing (Read)

  Details update (Update)

  Account deactivation (Delete).
- **Core Classes**:

  `User.java`

  `UserRepository.java`

  `UserService.java`

  `AuthController.java`

  `ProfileController.java`

  `SecurityConfig.java`

### 8.2 Module 2: Service & Package Management

- **Responsibility**: Maintenance of the salon's service catalog and promotional packages.
- **CRUD Operations**:

  Addition of new services/packages (Create)

  Catalog browsing and filtering (Read)

  Price/duration modification (Update)

  Service deprecation (Delete)
- **Core Classes**:

  `SalonService.java`

  `ServicePackage.java`

  `ServiceRepository.java`

  `PackageRepository.java`

  `ServiceService.java`

  `PackageService.java`

  `ServiceController.java`

  `PackageController.java`

### 8.3 Module 3: Staff / Beautician Management

- **Responsibility**: Employee record maintenance, specialty assignment, and availability tracking.
- **CRUD Operations**:
  Staff onboarding (Create)
  Employee directory search (Read)
  Schedule/specialty updates (Update)
  Staff offboarding (Delete).
- **Core Classes**:
  Staff.java
  Stylist.java
  Therapist.java
  StaffRepository.java
  StaffService.java
  StaffController.java

### 8.4 Module 4: Appointment Booking & Schedule

- **Responsibility**: The core transactional engine handling reservations and conflict resolution.
- **CRUD Operations**:
  Booking generation (Create)
  Schedule viewing (Read)
  Time/staff reassignment (Update)
  Booking cancellation (Delete).
- **Core Classes**:
  Appointment.java
  AppointmentRepository.java
  AppointmentService.java
  AppointmentController.java

### 8.5 Module 5: Admin Dashboard & Reports

- **Responsibility**: Centralized administrative oversight, KPI aggregation, and user role management.
- **CRUD Operations**:
  Admin provisioning (Create)
  Dashboard analytics viewing (Read)
  Role escalation (Update)
  System-wide account deletion (Delete).
- **Core Classes**:
  `Admin.java`
  `AdminController.java`
  `AdminService.java`
  `ReportService.java`
  `DashboardSummaryDTO.java`
  `SecurityConfig.java`

### 8.6 Module 6: Reviews & Post-Visit Flow

- **Responsibility**: Collection, display, and moderation of customer feedback.
- **CRUD Operations**:
  Review submission (Create)
  Public review aggregation (Read)
  Content modification/moderation status updates (Update)
  Removal of malicious feedback (Delete).
- **Core Classes**:
  `Review.java`
  `ReviewRepository.java`
  `ReviewService.java`
  `ReviewController.java`

## 9. Conclusion

The Sakura Saki Beauty Salon Management System stands as a comprehensive, successfully executed digital transformation project that bridges the gap between complex backend logic and a beautiful, engaging user interface. By adhering to a strict layered N-tier architecture and applying robust Object-Oriented Programming principles specifically Encapsulation, Inheritance, Abstraction, and Polymorphism the development team has delivered a highly scalable, secure, and maintainable enterprise application.

The clear delineation of responsibilities across the six specialized modules (Authentication, Services, Staff, Appointments, Admin Dashboard, and Reviews) ensured efficient parallel development and minimized integration bottlenecks. Furthermore, the integration of a modern DevOps pipeline utilizing Docker containerization, automated GitHub Actions for CI/CD and Render for cloud deployment demonstrates a commitment to industry-standard software engineering practices.

Ultimately, Sakura Saki not only fulfils its functional requirements by eliminating manual scheduling conflicts and centralizing salon management, but it also provides a premium, interactive 3D experience that delights customers. The resulting software solution is a professional grade, production-ready platform poised to significantly enhance operational efficiency, staff coordination, and customer satisfaction for any modern beauty salon.

## 10.Class Diagram

Drive URL Of Diagram - https://drive.google.com/drive/folders/1yMJpl0neiHbNOAaAv1GzhuPjDlxakU-f?usp=sharing

Website URL - https://sakura-saki.onrender.com/

GitHub Repository - https://github.com/CJChANu/Sakura_Saki.git